EECE 7205: Computing Fundamentals for Engineers

Fall 2026

In Class Work #03 Binary Search Trees

Topic: Deleting Nodes from Binary Search Trees

Student Name	Eduardo Scopelli
NUID	003972121
Email	scopellirivas.e@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	10-18-2026

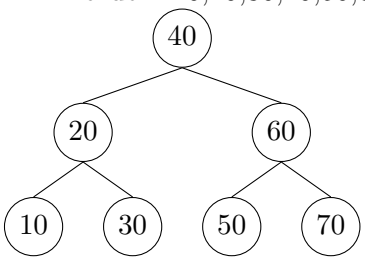
Checklist Item

Why deletion needs its own rules: In a binary search tree every value in the left subtree of a node is smaller than the node, and every value in its right subtree is larger. Deleting a node must keep this property for every node that stays in the tree. That is why we cannot simply move the deepest node into the gap, as we did for the level-order binary tree. Instead, we look at how many children the node has and pick one of the rules below.

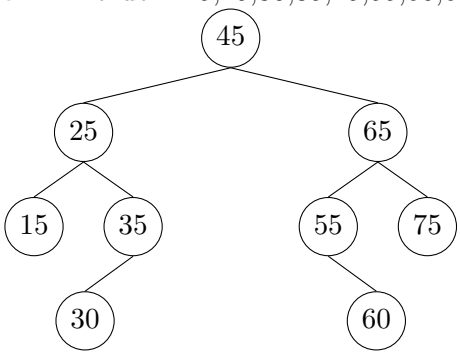
- Work in pairs.
- Draw first, run second.
- Use the provided programs.
- Show your work.

Rule	What it says
B1 Search	Start at the root and remember the parent. If key < node go left, if key > node go right.
B2 Not found	If the search reaches NULL, the key is not in the tree and nothing changes.
B3 Leaf	The node has no children: set the parent's link to NULL and free the node.
B4 One child	The only child takes the node's place: the parent now points to the child.
B5 Two children	Find the inorder successor (right once, then left as far as possible), copy its value into the node, then remove the successor with B3 or B4.
B6 Root	The root follows the same rules. With 0 or 1 child, the root pointer itself changes.
B7 Check	After every deletion, the inorder traversal must still be in increasing order.

Tree T: Inorder=10,20,30,40,50,60,70



Tree P: Inorder=15,25,30,35,45,55,60,65,75

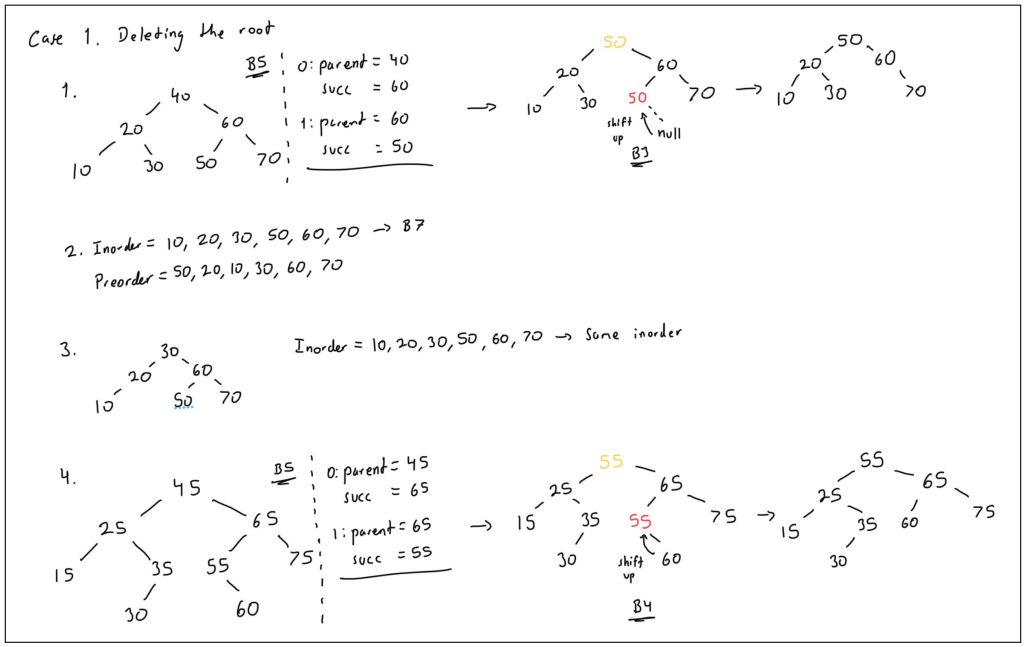


Case 1: Deleting the Root

Description: The root is the first node of the tree, so deleting it is the binary search tree version of "delete at the beginning". In tree T, the root 40 has two children, so we cannot just remove it: both subtrees would lose their parent. Instead, we find the inorder successor, the smallest value that is larger than 40. It is always in the right subtree: go right once (60), then left as far as possible (50). We copy 50 into the root and remove the old node 50, which is a leaf. The root keeps its two subtrees, and 50 is still larger than everything on the left and smaller than everything on the right.

Rules used: B5 and B6.

Notes file



Question 1

Instructions: In tree T, list every node visited while looking for the inorder successor of the root. Give the successor's value and the value of its parent.

As shown in the image above, the program goes through two iterations of the search loop before reaching the inorder successor of the root.

Iteration	Parent	Successor	Direction
0	40	60	right
1	**60**	**50**	left

Question 2

Instructions: Draw tree T after deleting 40. Write its inorder and preorder traversals, and explain how the inorder result shows that rule B7 still holds.



Inorder = 10, 20, 30, 50, 60, 70; **Preorder** = 50, 20, 10, 30, 60, 70

Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 1
Generated tree: 10 20 30 40 50 60 70
Case selection (Cases: 1 - 5, General deletion: 6): 1
Select a target: 0
Before: 10 20 30 40 50 60 70
After:  10 20 30 50 60 70
```

Question 3

Instructions: Modify case1.cpp to use the inorder predecessor instead: the largest value in the left subtree (go left once, then right as far as possible). What is the new root, and does the inorder output change? Why or why not?

```
void case_1_3(Node* node) {
    // Case 1: delete the root (40). It has two children.
    // 1. find the inorder predecessor: the largest value in the left subtree
    Node* parent ← node;
    Node* pred ← node->left;
    while (pred->right ≠ nullptr) {
        parent ← pred;
        pred ← pred->right;
    }
    // 2. copy the successor's value into the root
    node->data ← pred->data;
    // 3. remove the successor node (it has no left child)
```

```
    if (parent = node) parent->left ← pred->left;
    else parent->right ← pred->left;
    delete pred;
}
```

Root: 30; **Inorder:** 10, 20, 30, 50, 60, 70.

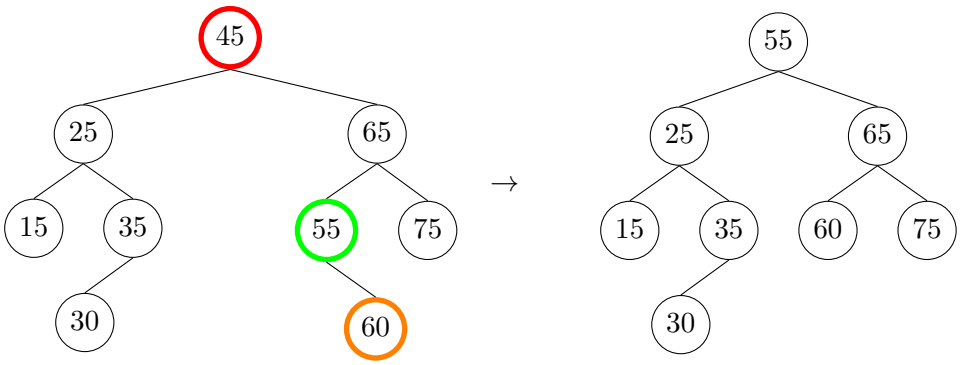
When switching between using the inorder successor or predecessor to delete the root of a tree, the inorder output is kept the same, despite the final tree being different. This is due to the fact that from the initial inorder output (10, 20, 30, 40, 50, 60, 70) this decision only establishes if every node value before the root shifts right once, or every node value after the root shifts left, keeping every value in order despite being different operations in the tree.

Question 4

Instructions: In tree P, delete the root 45. Which node is the successor? Which rule removes the successor node, and why is it not B3 this time? Draw the tree after the deletion.

Iteration	Parent	Successor	Direction
0	45	65	right
1	**65**	**55**	left

For this tree, we iterate the same number of times as in question 1 for tree P to get to the inorder successor. In this case, however, this successor has one right child instead of being a leaf, so in order to delete the root we need to first copy this inorder successor's value to the root, and before deleting the bottom node we must shift it's right child up to it's spot using rule B4 (so node 55 becomes the root, and node 60 takes the spot of node 55).



Code execution:

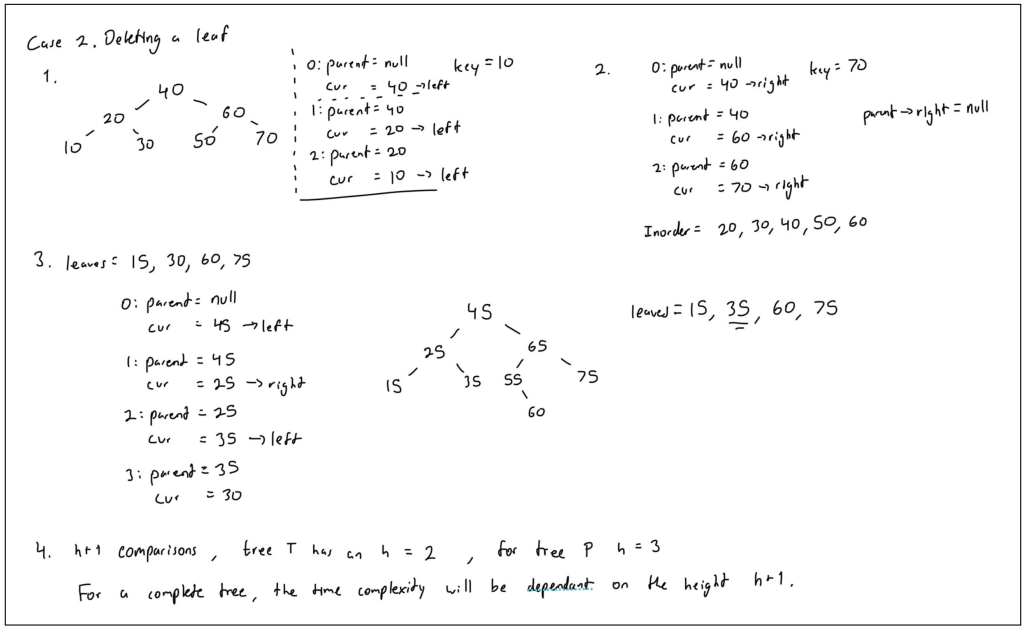
```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 2
Generated tree: 15 25 30 35 45 55 60 65 75
Case selection (Cases: 1 - 5, General deletion: 6): 1
Select a target: 0
Before: 15 25 30 35 45 55 60 65 75
After:  15 25 30 35 55 60 65 75
```

Case 2: Deleting a leaf

Description: A leaf has no children, so nothing hangs below it. Removing it cannot break the order of any other node. The only thing to do is find it, remember its parent while searching (rule B1), set the parent's link to NULL, and free the node. The program searches for 10: $10 < 40$ so it goes left, $10 < 20$ so it goes left again, and finds 10 with parent 20. Because 10 is 20's left child, it sets 20->left to NULL.

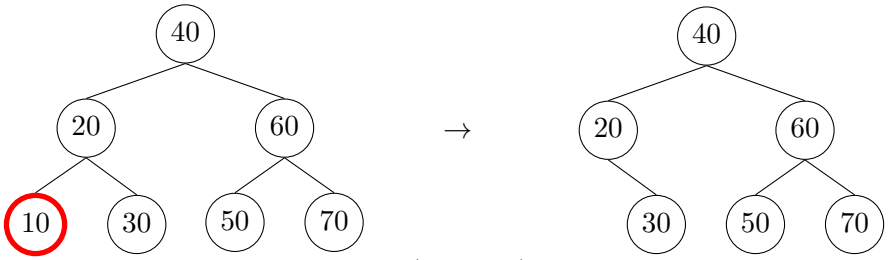
Rules used: B1 and B3.

Notes file



Question 1

Instructions: Trace the search loop for key = 10 in tree T. For each node visited, write the comparison made and the direction taken. Which node is the parent, and is the deleted node its left or its right child?



Node 20 is the parent, and it's left child (node 10) was deleted.

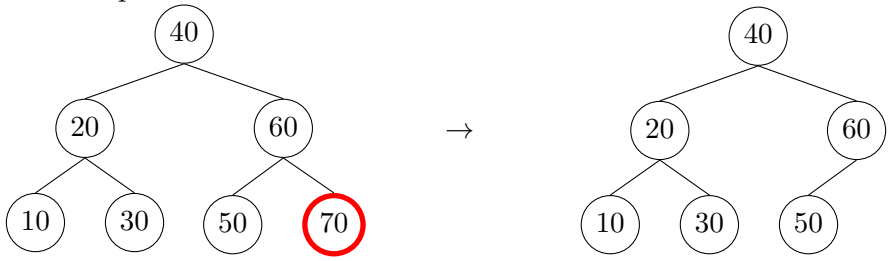
Iteration	Parent	Current	Comparison	Direction
0	null	40	$10 < 40$	left
1	40	20	$10 < 20$	left
2	**20**	**10**	–	–

Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 1
Generated tree: 10 20 30 40 50 60 70
Case selection (Cases: 1 - 5, General deletion: 6): 2
Select a target: 10
Before: 10 20 30 40 50 60 70
After:  20 30 40 50 60 70
```

Question 2

Instructions: Change the key to 70 and run the program again. Which pointer is set to NULL now, and what is printed after the deletion?



Pointer $60 \rightarrow right = null$, **Inorder** = 10, 20, 30, 40, 50, 60

Iteration	Parent	Current	Comparison	Direction
0	null	40	$70 > 40$	right
1	40	60	$70 > 60$	right
2	**60**	**70**	–	–

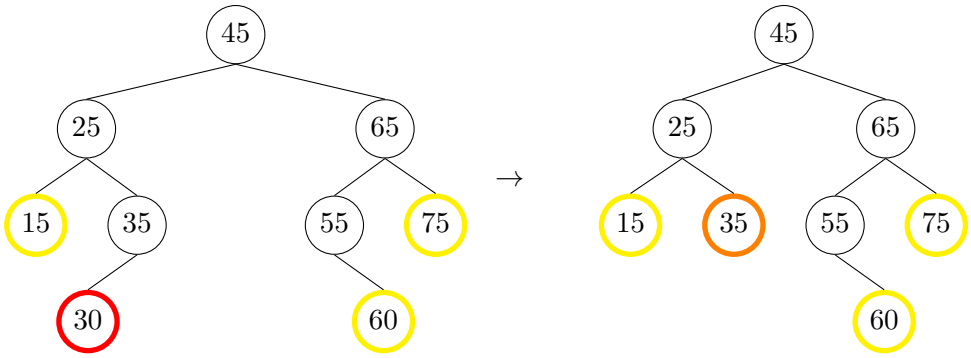
Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 1
Generated tree: 10 20 30 40 50 60 70
Case selection (Cases: 1 - 5, General deletion: 6): 2
Select a target: 70
Before: 10 20 30 40 50 60 70
After:  10 20 30 40 50 60
```

Question 3

Instructions: List all the leaves of tree P. Delete 30 from P, draw the tree after the deletion, and name the node that has just become a leaf.

Tree P has 4 leaves: 15, 30, 60 and 75.



New leaf: Node 35

Iteration	Parent	Current	Comparison	Direction
0	null	45	$30 < 45$	left
1	45	25	$30 > 25$	right
2	25	35	$30 < 35$	left
3	**35**	**30**	–	–

Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 2
Generated tree: 15 25 30 35 45 55 60 65 75
Case selection (Cases: 1 - 5, General deletion: 6): 2
Select a target: 30
Before: 15 25 30 35 45 55 60 65 75
After:  15 25 35 45 55 60 65 75
```

Question 4

Instructions: How many key comparisons does the search make in the worst case for a tree of height h? What is h for tree T, and what is h for the tree you get by inserting 10, 20, 30, 40, 50, 60, 70 in that order? What does this tell you about the time to delete a leaf?

In a tree of height h ($h + 1$ levels), the search loop makes $2h + 1$ comparisons to reach the final level: h comparisons are completed when checking if the current node is greater than or less than the target before reaching the target in the final level, and the remaining $h + 1$ comparisons occur in the while loop to iterate over each level, with one additional comparison to exit the while loop on the last level.

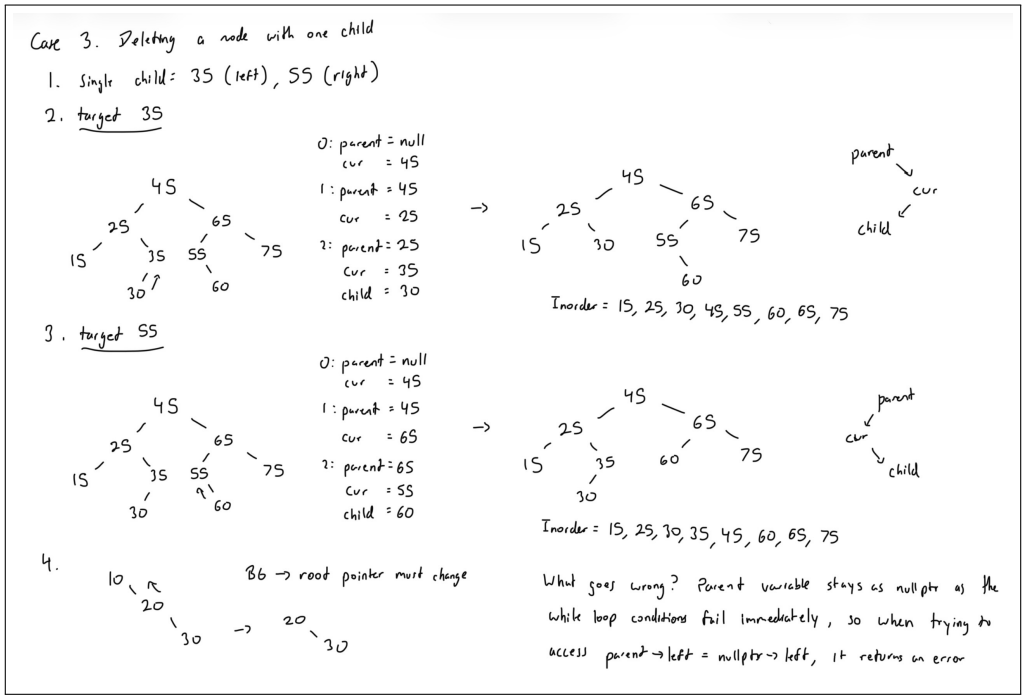
Tree T has a height of $h = 2$, while a tree with inserted nodes of 10, 20, 30, 40, 50, 60, 70 in that order has a height $h = 6$ with 10 being the root and all the following nodes connecting to the right.

Case 3: Deleting a node with one child

Description: When a node has exactly one child, that child (together with everything below it) simply moves up into the node's place. This is safe because the whole child subtree was already on the correct side of the parent: if the node was the parent's right child, every value below it is larger than the parent, and that is still true after it moves up. In tree T every node has 0 or 2 children, so case3.cpp first removes 50 to give 60 a single child, 70. Then it deletes 60 and links 40->right directly to 70.

Rules used: B1 and B4.

Notes file

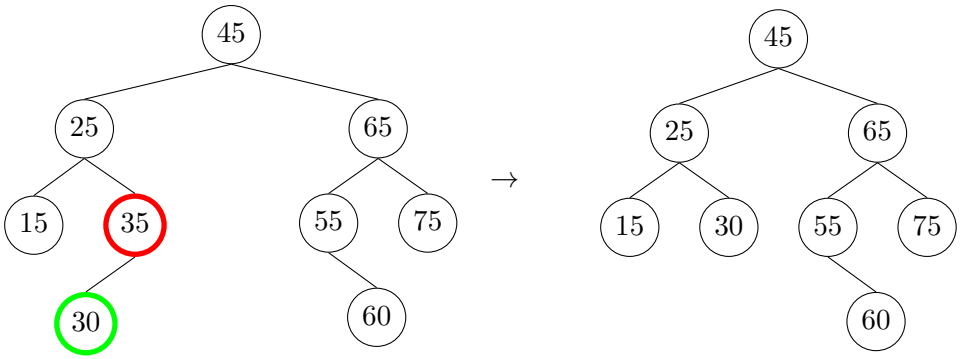


Question 1

Instructions: Which nodes of tree P have exactly one child? For each one, say whether the child is a left or a right child.
Single child nodes: 35 (has a left child 30) and 55 (has a right child 60).

Question 2

Instructions: Delete 35 from P. Name the parent, the child that moves up, and the exact pointer that changes. Draw the tree and write its inorder traversal.



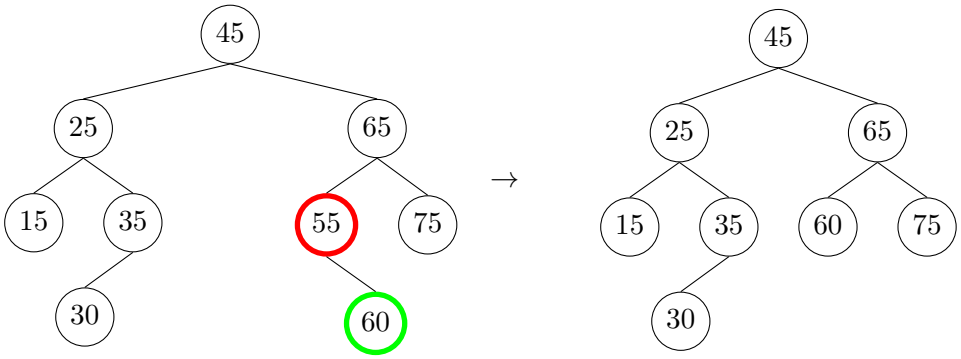
Iteration	Parent	Current	Comparison	Direction
0	null	45	$35 < 45$	left
1	45	25	$35 > 25$	right
2	**25**	**35**	–	–

Node 35 gets deleted, so node 30 moves up and is linked to it's new parent, node 25 ($25 \rightarrow right = 30$).
Code execution:

```
● scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
  Tree selection (T: 1, P: 2): 2
  Generated tree: 15 25 30 35 45 55 60 65 75
  Case selection (Cases: 1 - 5, General deletion: 6): 3
  Select a target: 35
  Before: 15 25 30 35 45 55 60 65 75
  After:  15 25 30 45 55 60 65 75
```

Question 3

Instructions: Start again from the original P and delete 55. Answer the same three parts as in question 2. Why does the program use parent->left this time instead of parent->right?



Iteration	Parent	Current	Comparison	Direction
0	null	45	$55 > 45$	left
1	45	65	$55 < 65$	right
2	**65**	**55**	–	–

Node 55 gets deleted, so node 60 moves up and is linked to it's new parent, node 65 ($65 \rightarrow left = 60$). In this case, as the node deleted was the left child of 65, the code redefines the left child pointer $65 \rightarrow left$ instead of a right child pointer as in the previous question.

Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 2
Generated tree: 15 25 30 35 45 55 60 65 75
Case selection (Cases: 1 - 5, General deletion: 6): 3
Select a target: 55
Before: 15 25 30 35 45 55 60 65 75
After:  15 25 30 35 45 60 65 75
```

Question 4

Instructions: Build a tree by inserting 10, 20, 30 in that order and delete the root 10. Which rule tells you that the root pointer itself must change? What is the new root, and what goes wrong in case3.cpp if you use it unchanged (hint: what is parent when the root is deleted)?

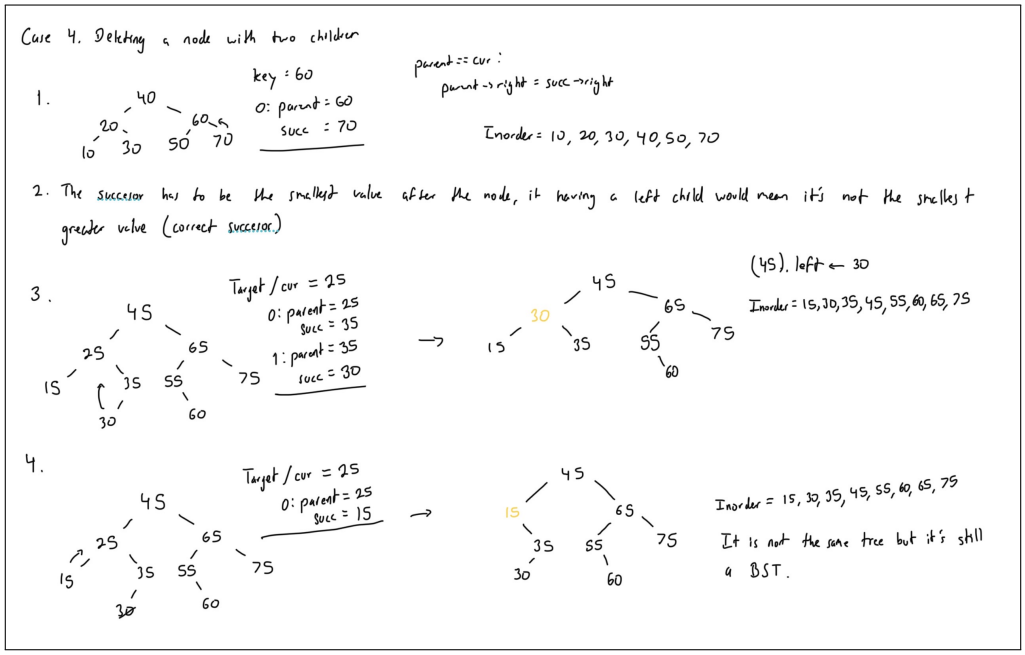
For this question, deleting the root 10 requires the use of rule B6, where the root pointer must change as the root is being replaced by it's child. As shown in the previous notes image, the tree will go from a three node tree with nodes 10, 20 and 30 connected sequentially to one with nodes 20 and 30, with 20 being the new root. Unfortunately, with the code provided this deletion will result in an error due to the fact that, as our target node is our root, the code never completes a single search iteration, which means that the value of the parent variable (which we keep to connect the parent of the target node to the children of the target node) stays as null. At the end of the program when it tries to access and redefine the child pointer of the parent node, it will return an error as this variable isn't initialized as a node yet.

Case 4: Deleting a node with two children

Description: A node with two children cannot be replaced by one of them, because the other subtree would lose its place. We use the same idea as in Case 1: copy in the value of the inorder successor and then delete the successor node instead. To delete 20 in tree T, the successor is found by going right once (30) and then left as far as possible (30 has no left child, so we stop). 30 is copied into the node, and the old 30 is removed. Because the successor is the leftmost node of a subtree, it never has a left child, so removing it is always an easy B3 or B4 case.

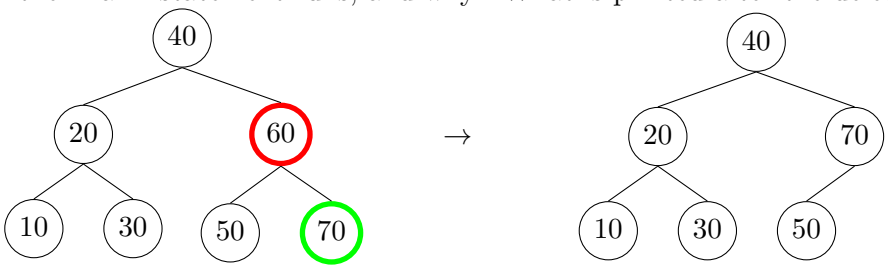
Rules used: B1 and B5 (then B3 or B4 for the successor).

Notes file



Question 1

Instructions: Change key to 60 in case4.cpp. What are succ and parent after the loop? Which branch of the final if statement runs, and why? What is printed after the deletion?



Successor: 70, **Parent:** 60, **Inorder** = 10, 20, 30, 40, 50, 70.
In the final if statement, the condition *parent* == *cur* is fulfilled as the inorder successor is the immediate right child of the target, so the first branch is executed, setting the parent's right pointer as the successor's right child (null) before deleting this successor.

Iteration	Parent	Successor	Comparison	Direction
1	**60**	**70**	–	–

Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 1
Generated tree: 10 20 30 40 50 60 70
Case selection (Cases: 1 - 5, General deletion: 6): 4
Select a target: 60
Before: 10 20 30 40 50 60 70
After:  10 20 30 40 50 70
```

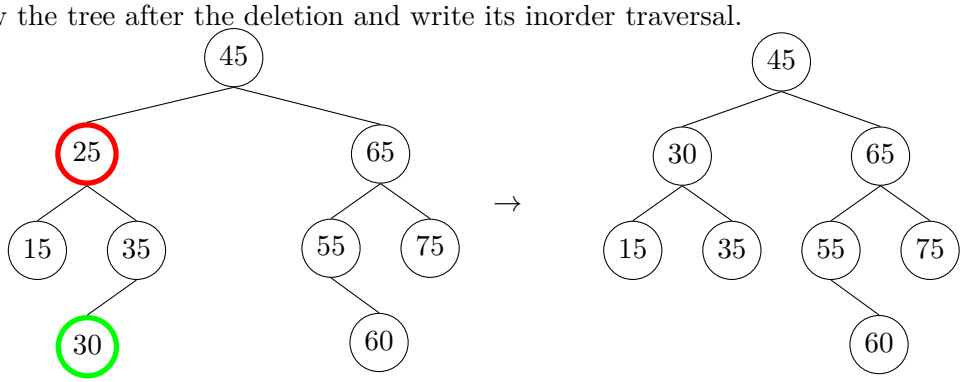
Question 2

Instructions: Explain in your own words why the inorder successor of a node with two children can never have a left child. What would go wrong in step 4 of the code if it could?

The successor has to be the smallest value after the target node, so having an inorder successor to the target node with a left child simply means that the left child is the real inorder successor (unless there are other left children in sequence to this one). In terms of an inorder output, this simply means that there is still a value after our target node but before the currently identified successor.

Question 3

Instructions: In tree P, delete 25. Give the successor, its parent, and the pointer that is changed. Draw the tree after the deletion and write its inorder traversal.



Iteration	Parent	Successor	Comparison	Direction
0	25	35	35→ $left \neq$null	left
1	**35**	**30**	30→ $left =$null	–

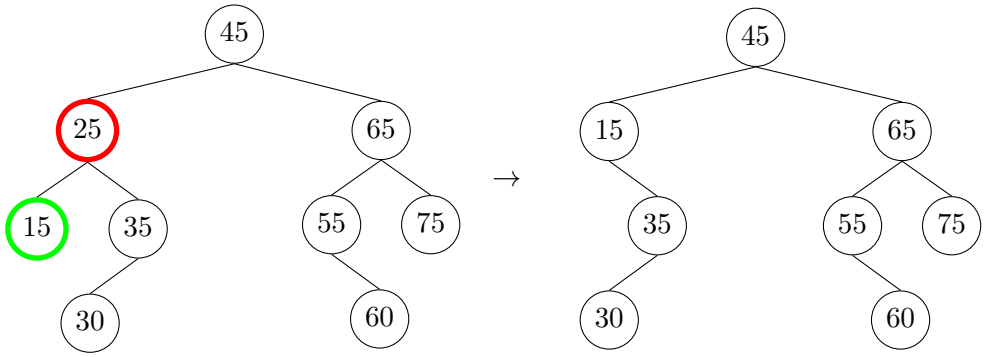
Node 25 gets deleted, the successor 30 with parent 30 replaces this deleted node such that $45 \rightarrow left = 30$. In this case, as the successor wasn't the immediate right child of the target, the program uses the *else* branch of the last section of the program, setting the left child of the parent to the right child of the successor. The inorder output is: Inorder = 15,30,35,45,55,60,65,75.

Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 2
Generated tree: 15 25 30 35 45 55 60 65 75
Case selection (Cases: 1 - 5, General deletion: 6): 4
Select a target: 25
Before: 15 25 30 35 45 55 60 65 75
After:  15 30 35 45 55 60 65 75
```

Question 4

Instructions: Delete 25 from the original P again, but this time use the inorder predecessor. Draw the result. Is it the same tree as in question 3? Are both valid binary search trees?



Iteration	Parent	Predecessor	Comparison	Direction
0	**25**	**15**	15→ *right* =null	–

Node 25 gets deleted, the predecessor 15 with parent 25 replaces this deleted node such that $45 \rightarrow left = 15$. In this case, as the predecessor was the immediate left child of the target, the program uses the if branch of the last section of the program, setting the left child of the parent to the left child of the successor. Compared to the previous result, we end up with two different trees, but both trees have the same inorder output (Inorder = 15,30,35,45,55,60,65,75), which means that both trees are valid binary search trees.

Code execution:

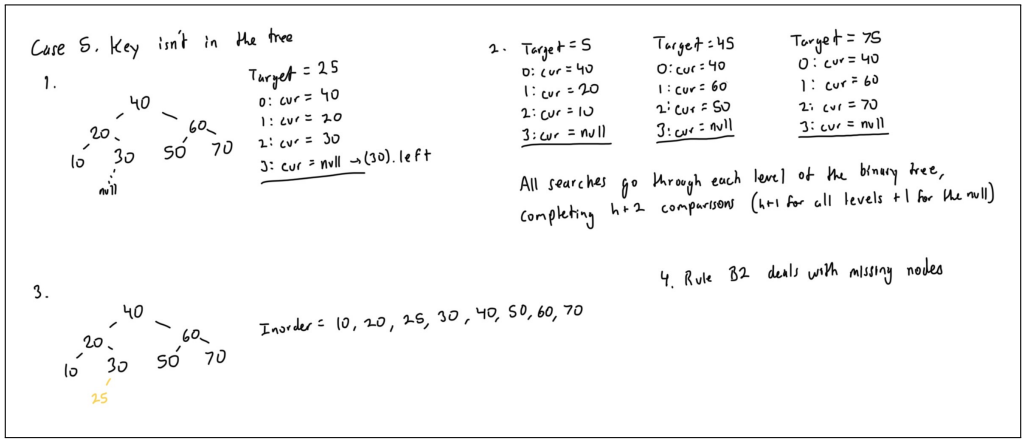
```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 2
Generated tree: 15 25 30 35 45 55 60 65 75
Case selection (Cases: 1 - 5, General deletion: 6): 8
Select a target: 25
Before: 15 25 30 35 45 55 60 65 75
After:  15 30 35 45 55 60 65 75
```

Case 5: Key is not in the tree

Description: Before deleting anything we must find the key. The search follows the BST property: at each node it goes left if the key is smaller and right if it is larger. If it reaches NULL, the key is not in the tree and the tree must be left exactly as it was. For 25 in tree T the path is 40 (go left), 20 (go right), 30 (go left), then NULL. The NULL position the search stops at is exactly where 25 would go if we inserted it, which is why insertion and search share the same loop.

Rules used: B1 and B2.

Notes file



Question 1

Instructions: Trace the search for 25. How many comparisons with node values does it make, and at which NULL link (which node, left or right) does it stop?

Iteration	Current	Direction
0	40	left
1	20	right
2	30	left
3	null	–

Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 1
Generated tree: 10 20 30 40 50 60 70
Case selection (Cases: 1 - 5, General deletion: 6): 5
Select a target: 25
Before: 10 20 30 40 50 60 70
25 is not in the tree: nothing to delete
After:  10 20 30 40 50 60 70
```

This algorithm completes $h + 1$ comparisons between the target value and each node stored in the current variable. Additionally, the while loop completes $h + 2$ additional checks by checking that neither the target has been found and the current node isn't null, which gives a total of $2h + 3$ for the entire search loop, stopping at the left child of the leaf 30, which gives a value of null.

Question 2

Instructions: Do the same for the keys 5, 45 and 75. Which one needs the most comparisons, and how does that relate to the height of the tree?

Target: 5

Iteration	Current	Direction
0	40	left
1	20	left
2	10	left
3	null	–

Target: 45

Iteration	Current	Direction
0	40	right
1	60	left
2	50	left
3	null	–

Target: 75

Iteration	Current	Direction
0	40	right
0	60	right
0	70	right
0	null	–

In the end, all of these cases complete the same amount of comparisons and iterations, as each one of them reaches the last level of our tree of height $h = 2$ due to the fact that we have a tree with it's last level being complete, so no matter which value we search for, it will always go through this last level if it isn't in the tree. For any other tree, the amount of comparisons and iterations for searching for a missing node will fall in between the shortest depth from the root and the height of the tree.

Question 3

Instructions: Modify case5.cpp so that when the key is not found, it inserts the key at the NULL link where the search stopped (you will need to remember the parent). Insert 25 and print the inorder traversal.

```
void case_5_3(Node* node, int target) {
    // Case 5: the key is not in the tree.
    if (!node) {
        cout << "Tree is empty" << endl;
        return;
    }
    Node* parent ← nullptr;
    Node* cur ← node;
```

```
    int key ← target;
    while (cur ≠ nullptr && cur->data ≠ key) {
        parent ← cur;
        if (key < cur->data) cur ← cur->left;
        else cur ← cur->right;
    }
    if (cur = nullptr)
        if (parent->data > target) {
            parent->left ← new Node(target);
        } else {
            parent->right ← new Node(target);
        }
}
```

Inorder: 10, 20, 25, 30, 40, 50, 60, 70.

Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 1
Generated tree: 10 20 30 40 50 60 70
Case selection (Cases: 1 - 5, General deletion: 6): 9
Select a target: 25
Before: 10 20 30 40 50 60 70
After:  10 20 25 30 40 50 60 70
```

Question 4

Instructions: case5.cpp prints root->data. What happens if the tree is empty (root = nullptr)? Add a check so the program prints "the tree is empty" instead. Which rule covers this case?
When the program tries to access the value of root->data, it will generate an error as the variable root will be null, not a node, thus it has no attribute data. This can be solved with a simple check at the beginning of the function.

Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 3
Generated tree:
Case selection (Cases: 1 - 5, General deletion: 6): 9
Select a target: 20
Before:
Tree is empty
After:
```

Bonus: One function for every case

Description: Combine the five programs into one function Node* deleteNode(Node* root, int key) that returns the (possibly new) root and handles every case: not found, leaf, one child, two children and the root. Test it on tree T by deleting 40, 10, 60 and 25 one after another, calling printInorder after each deletion. State the time complexity in terms of h.

Code

```
void deleteNode(Node* node, int target) {
    Node* curr ← node;
    Node* prev ← nullptr;
    while (curr ≠ nullptr && curr->data ≠ target) {
        prev ← curr;
        if (target < curr->data) curr ← curr->left;
        else curr ← curr->right;
    }
    if (curr = nullptr) {
        if (prev->data > target) prev->left ← new Node(target);
        else prev->right ← new Node(target);
        return;
    }
    if (curr->left = nullptr || curr->right = nullptr) {
        Node* newChild ← (curr->left ≠ nullptr) ? curr->left : curr->right;
        if (prev = nullptr) {
            delete curr;
            node ← newChild;
            return;
        }
        if (curr = prev->left) prev->left ← newChild;
        else prev->right ← newChild;
        delete curr;
    }
    else {
        Node* succParent ← curr;
        Node* succ ← curr->right;
        while (succ->left ≠ nullptr) {
            succParent ← succ;
            succ ← succ->left;
        }
        curr->data ← succ->data;
        if (succParent->left = succ) succParent->left ← succ->right;
        else succParent->right ← succ->right;
        delete succ;
    }
}
```

Test cases

Target: 40
Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 1
Generated tree: 10 20 30 40 50 60 70
Case selection (Cases: 1 - 5, General deletion: 6): 6
Select a target: 40
Before: 10 20 30 40 50 60 70
After:  10 20 30 50 60 70
```

Target: 10
Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 1
Generated tree: 10 20 30 40 50 60 70
Case selection (Cases: 1 - 5, General deletion: 6): 6
Select a target: 10
Before: 10 20 30 40 50 60 70
After:  20 30 40 50 60 70
```

Target: 60
Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 1
Generated tree: 10 20 30 40 50 60 70
Case selection (Cases: 1 - 5, General deletion: 6): 6
Select a target: 60
Before: 10 20 30 40 50 60 70
After:  10 20 30 40 50 70
```

Target: 25
Code execution:

```
scopellie@DESKTOP-0MDARFH:~/EECE7205/Class_work/Classwork02$ /home/scopellie/EECE7205/Class_work/Classwork02/BST_Classwork
Tree selection (T: 1, P: 2): 1
Generated tree: 10 20 30 40 50 60 70
Case selection (Cases: 1 - 5, General deletion: 6): 6
Select a target: 25
Before: 10 20 30 40 50 60 70
After:  10 20 25 30 40 50 60 70
```